

UNIwersYTET RZESZOWSKI
WYDZIAŁ NAUK ŚCISŁYCH I TECHNICZNYCH
INSTYTUT INFORMATYKI



Piotr Pająk, Mateusz Czapór
131483, 131416

Informatyka

*Projekt Zaliczeniowy: System biometryczny oparty na
rozpoznawaniu twarzy i analizie emocji*

Projekt zaliczeniowy

Prowadzący
dr Zbigniew Gomółka

Rzeszów 2025

Spis treści

1. Wstęp.....	6
2. Architektura, technologie i obsługa systemu.....	7
2.1. Wykorzystane technologie.....	7
2.2. Struktura modułowa	7
2.3. Instrukcja uruchomienia i obsługi	8
2.3.1. Instalacja	8
2.3.2. Uruchomienie i wybór kamery	9
2.3.3. Interfejs główny i rejestracja użytkownika	9
3. Autorska implementacja	11
3.1. Zarządzanie sesją i danymi biometrycznymi.....	11
3.2. Mechanizmy rozpoznawania i optymalizacji	11
3.2.1. Identyfikacja użytkownika i metryki dopasowania.....	11
3.2.2. Optymalizacja wydajności poprzez śledzenie obiektów.....	12
3.3. Autorski algorytm analizy emocji	13
3.3.1. Kalibracja spersonalizowanego modelu.....	13
3.3.2. Geometryczna analiza cech.....	13
3.3.3. Wnioskowanie oparte na regułach i progach	14
3.3.4. Implementacja analizy geometrycznej.....	14
3.3.5. Wątek przechwytywania wideo	15
3.3.6. Rejestracja nowych użytkowników.....	16
4. Wyniki i analiza.....	19
4.1. Skuteczność autorskiego systemu zarządzania danymi.....	19
4.2. Analiza wydajności jako efekt inteligentnego zarządzania sesją	19
5. Podsumowanie.....	21
Bibliografia	22
Spis rysunków	23
Spis tabel	24
Spis listingów	24
Oświadczenie studenta o samodzielności pracy	25
Oświadczenie studenta o samodzielności pracy	26

1. Wstęp

Celem niniejszego projektu było zaprojektowanie i zaimplementowanie zaawansowanego systemu biometrycznego czasu rzeczywistego, który integruje dwie kluczowe technologie: rozpoznawanie twarzy oraz analizę emocji. System został opracowany w języku Python z wykorzystaniem bibliotek open-source, jednak kluczowe komponenty, takie jak logika biznesowa, zarządzanie danymi oraz algorytmy interpretacji cech biometrycznych, stanowią autorską implementację.

Projekt wpisuje się w obszar zagadnień związanych z bezpieczeństwem systemów informatycznych oraz interakcją człowiek-komputer (HCI). Główne założenia funkcjonalne systemu obejmują:

- Uwierzytelnianie użytkowników na podstawie unikalnych cech ich twarzy.
- Analizę mikroekspresji w celu określenia konkretnej emocji użytkownika w czasie rzeczywistym.
- Stworzenie spersonalizowanego modelu biometrycznego dla każdego użytkownika poprzez mechanizm kalibracji wykrywania emocji.
- Zapewnienie wysokiej wydajności działania dzięki zastosowaniu technik optymalizacyjnych.

2. Architektura, technologie i obsługa systemu

System został zaprojektowany w architekturze modułowej, co zapewnia jego elastyczność i łatwość w dalszym rozwoju. Składa się z trzech głównych warstw: interfejsu użytkownika (UI), logiki biznesowej oraz warstwy przetwarzania danych. Poniżej przedstawiono kluczowe technologie i komponenty wykorzystane w projekcie.

2.1. Wykorzystane technologie

Do budowy systemu wykorzystano następujące narzędzia i biblioteki:

- **Python 3.8:** Główny język programowania projektu.
- **OpenCV (Open Source Computer Vision Library):** Kluczowa biblioteka do przetwarzania obrazu w czasie rzeczywistym, przechwytywania klatek z kamery oraz podstawowych operacji na obrazie [3].
- **GTK 3 (GIMP Toolkit):** Wykorzystane do stworzenia graficznego interfejsu użytkownika (GUI) aplikacji. Zapewnia natywny wygląd i responsywność na różnych systemach operacyjnych.
- **dlib:** Biblioteka zawierająca zaawansowane algorytmy uczenia maszynowego. W projekcie używana do detekcji punktów charakterystycznych twarzy (facial landmarks) za pomocą pre-trenowanego modelu. Model `ten, shape_predictor_68_face_landmarks.dat`, jest dostępny publicznie i został stworzony przez autorów biblioteki [1]. Można go pobrać ze strony: http://dlib.net/files/shape_predictor_68_face_landmarks.dat.bz2.
- **face_recognition:** Biblioteka wysokiego poziomu, bazująca na dlib, używana do kodowania cech twarzy (face embeddings) i ich porównywania. Umożliwia identyfikację osób.
- **MediaPipe:** Framework od Google do budowania potoków przetwarzania danych percepcyjnych. W projekcie wykorzystano go do wydajnej detekcji siatki twarzy (face mesh), która stanowi podstawę dla autorskiego algorytmu analizy emocji [2].

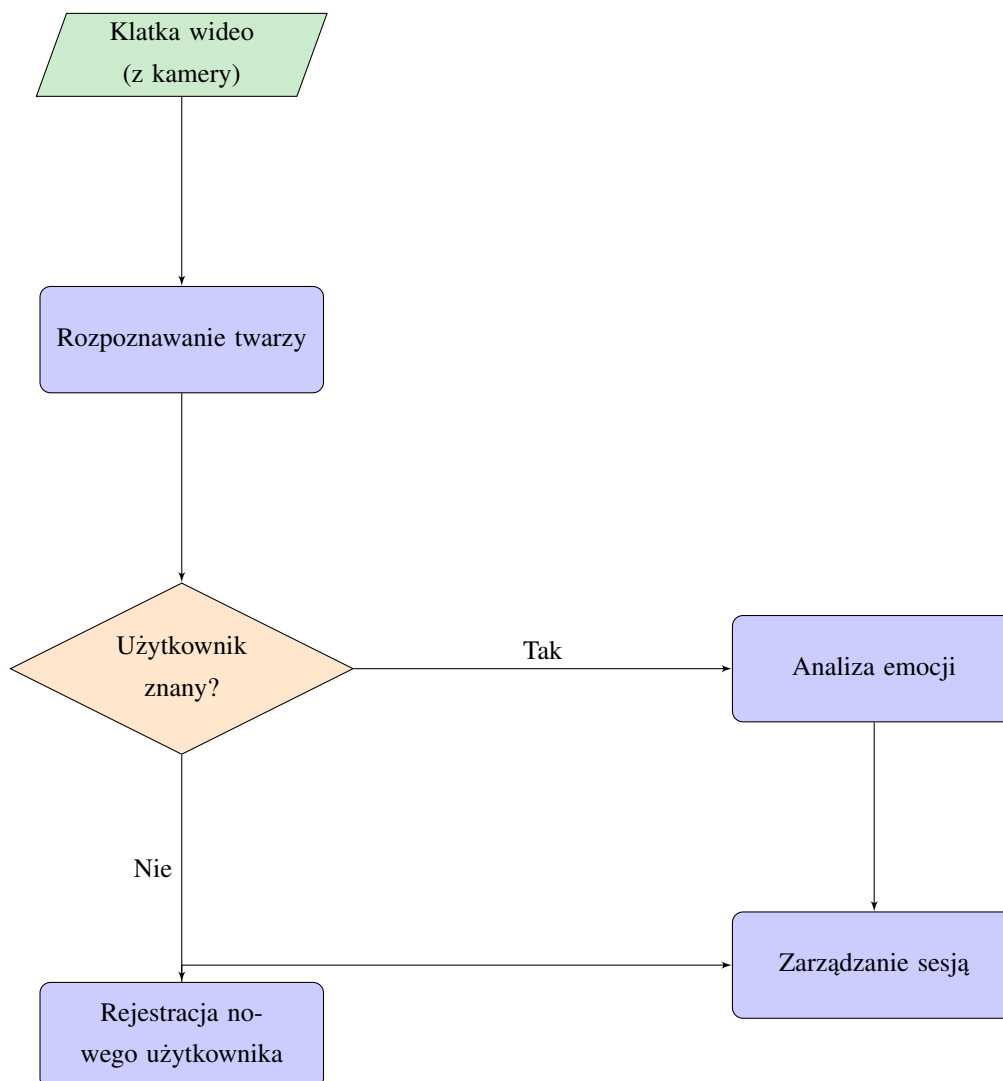
2.2. Struktura modułowa

Kod źródłowy został podzielony na logiczne moduły, co ułatwia zarządzanie projektem (szczegółowe listingi wybranych fragmentów znajdują się w rozdziale 3):

- `main.py`: Główny plik aplikacji, odpowiedzialny za inicjalizację interfejsu GTK, zarządzanie wątkiem kamery oraz obsługę zdarzeń.
- `biometric_system.py`: Zawiera klasę `BiometricSystem`, która zrzędza działaniem pozostałych komponentów, zarządza sesjami użytkowników i implementuje główną logikę biznesową.
- `face_utils.py`: Moduł odpowiedzialny za operacje związane z rozpoznawaniem twarzy, takie jak kodowanie i porównywanie wektorów cech.
- `emotion_analyzer.py`: Kluczowy moduł zawierający autorską implementację algorytmu do analizy emocji na podstawie geometrycznych relacji między punktami charakterystycznymi twarzy.

- `config.py`: Plik konfiguracyjny, przechowujący stałe i ustawienia aplikacji.

Na rysunku 2.1 przedstawiono schemat blokowy architektury systemu.



Rys. 2.1. Schemat blokowy architektury systemu biometrycznego.

2.3. Instrukcja uruchomienia i obsługi

Poniższe kroki opisują, jak przygotować środowisko oraz uruchomić aplikację biometryczną.

2.3.1. Instalacja

1. Tworzenie i aktywacja wirtualnego środowiska:

```
python -m venv venv
source venv/bin/activate    # Systemy Linux / macOS
venv\Scripts\activate.bat  # System Windows
```

2. Instalacja zależności (wymagany Python 3.8):

```
python3.8 -m pip install -r requirements.txt
```

3. **Pobranie modelu dlib:** Algorytm rozpoznawania punktów charakterystycznych twarzy wymaga pliku z wytrenowanym modelem. Należy go pobrać i umieścić w głównym katalogu projektu.

```
# W systemie Linux można użyć poniższych komend:  
wget http://dlib.net/files/shape_predictor_68_face_landmarks.dat.bz2  
bunzip2 shape_predictor_68_face_landmarks.dat.bz2
```

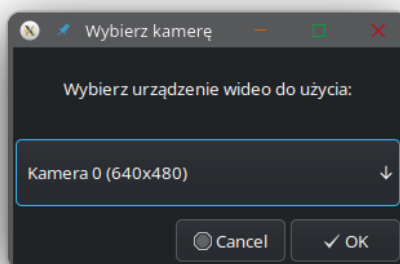
W systemie Windows należy pobrać plik ręcznie z adresu http://dlib.net/files/shape_predictor_68_face_landmarks.dat.bz2 i wypakować go za pomocą odpowiedniego programu (np. 7-Zip).

2.3.2. Uruchomienie i wybór kamery

Po poprawnej instalacji aplikację uruchamia się poleceniem:

```
python3.8 main.py
```

Przy pierwszym uruchomieniu lub gdy dostępna jest więcej niż jedna kamera, aplikacja wyświetli okno wyboru źródła wideo (pokazane na rys. 2.2). Należy wybrać odpowiednie urządzenie z listy i zatwierdzić przyciskiem "OK". Wybór zostanie zapisany w pliku `config.json` i będzie używany automatycznie przy kolejnych uruchomieniach.



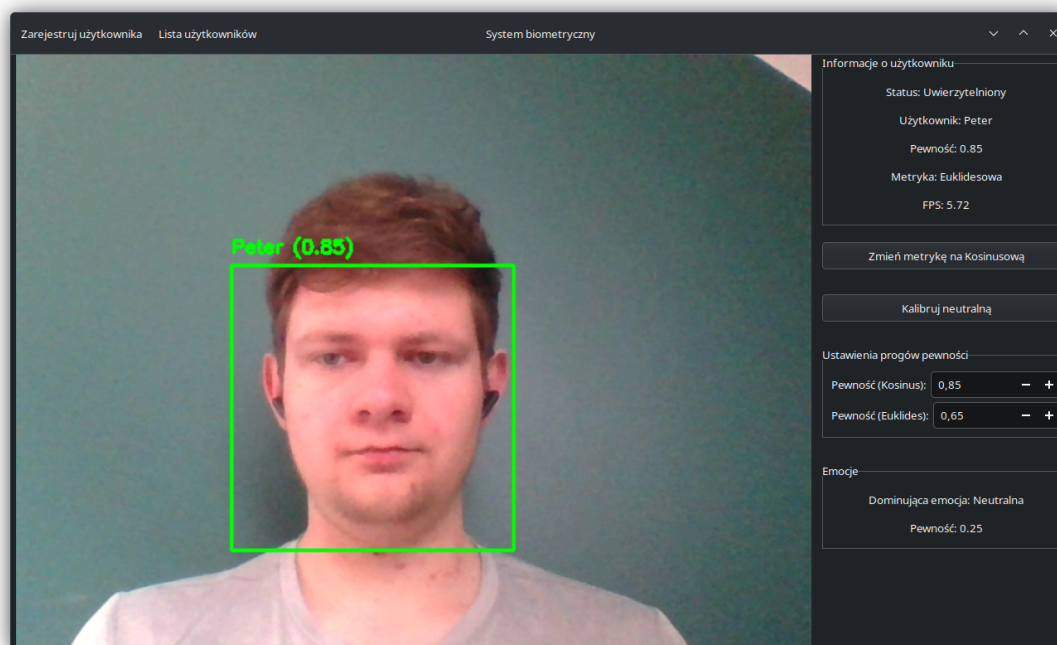
Rys. 2.2. Okno dialogowe wyboru kamery.

2.3.3. Interfejs główny i rejestracja użytkownika

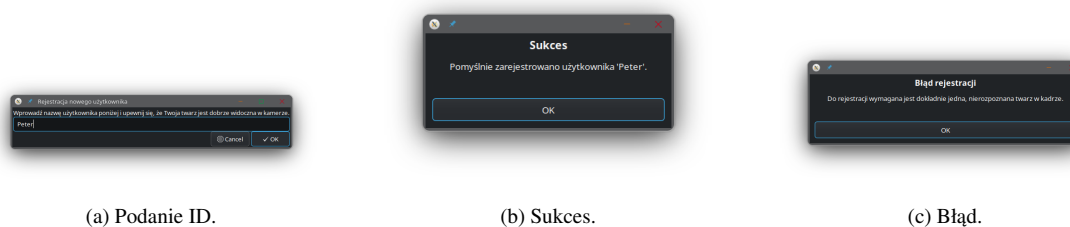
Interfejs główny aplikacji (rys. 2.3) składa się z okna podglądu wideo oraz panelu bocznego z informacjami i przyciskami.

Rejestracja nowej twarzy jest kluczowym krokiem przed rozpoczęciem korzystania z systemu. Aby zarejestrować użytkownika, należy:

1. Kliknąć przycisk *Register new user*.
2. W oknie dialogowym (rys. 2.4a) podać unikalny identyfikator (nazwę) użytkownika i zatwierdzić.
3. Spojrzeć prosto w kamerę, upewniając się, że twarz jest dobrze oświetlona i w całości widoczna. System automatycznie wykona zdjęcie i przetworzy je.
4. System poinformuje o wyniku operacji za pomocą odpowiedniego komunikatu (rys. 2.4b oraz 2.4c). W przypadku niepowodzenia, należy ponowić próbę, zwracając uwagę na lepsze warunki oświetleniowe. Należy pamiętać, że podczas rejestracji w kadrze może znajdować się tylko jedna twarz, a ponowne zarejestrowanie istniejącego użytkownika nie jest możliwe.



Rys. 2.3. Główny interfejs aplikacji z zidentyfikowanym użytkownikiem.



Rys. 2.4. Proces rejestracji nowego użytkownika.

3. Autorska implementacja

Najważniejszym elementem projektu, jest autorska implementacja kluczowych mechanizmów, które wykraczają poza standardowe użycie gotowych bibliotek. Poniżej opisano najważniejsze z nich.

3.1. Zarządzanie sesją i danymi biometrycznymi

Zaprojektowano i zaimplementowano własny system zarządzania danymi użytkowników. Służy do tego klasa `UserSession` w module `biometric_system.py`. Przechowuje ona kompleksowe informacje o stanie interakcji użytkownika z systemem, takie jak:

- Identyfikator użytkownika i jego stan uwierzytelnienia (np. zalogowany, w trakcie logowania).
- Historię wykrytych emocji, co pozwala na analizę trendów w zachowaniu.
- Spersonalizowaną bazę biometryczną (baseline) dla neutralnego wyrazu twarzy.

Stworzono również własny mechanizm serializacji i deserializacji obiektów `UserSession` do formatu JSON. Dzięki temu stan systemu jest trwale zapisywany na dysku i może być odtworzony po ponownym uruchomieniu aplikacji.

3.2. Mechanizmy rozpoznawania i optymalizacji

3.2.1. Identyfikacja użytkownika i metryki dopasowania

Proces identyfikacji opiera się na popularnej bibliotece `face_recognition`, która przekształca obraz twarzy na 128-wymiarowy wektor cech (tzw. *face embedding*). Podczas rejestracji wektor ten jest obliczany i zapisywany w bazie danych.

Podczas uwierzytelniania system porównuje wektor cech twarzy z bieżącej klatki ze wszystkimi zapisanymi wektorami. Do pomiaru podobieństwa zaimplementowano dwie metryki odległości:

- **Odległość euklidesowa** — standardowa, prosta odległość między dwoma punktami w przestrzeni wielowymiarowej. Jest intuicyjna, ale może być wrażliwa na zmiany w jasności obrazu.
- **Podobieństwo kosinusowe** — oblicza kosinus kąta między dwoma wektorami. Jest mniej wrażliwe na magnitudę wektorów, a bardziej na ich orientację, co czyni je bardziej odpornym na zmiany w oświetleniu. Domyślnie system korzysta z tej metryki.

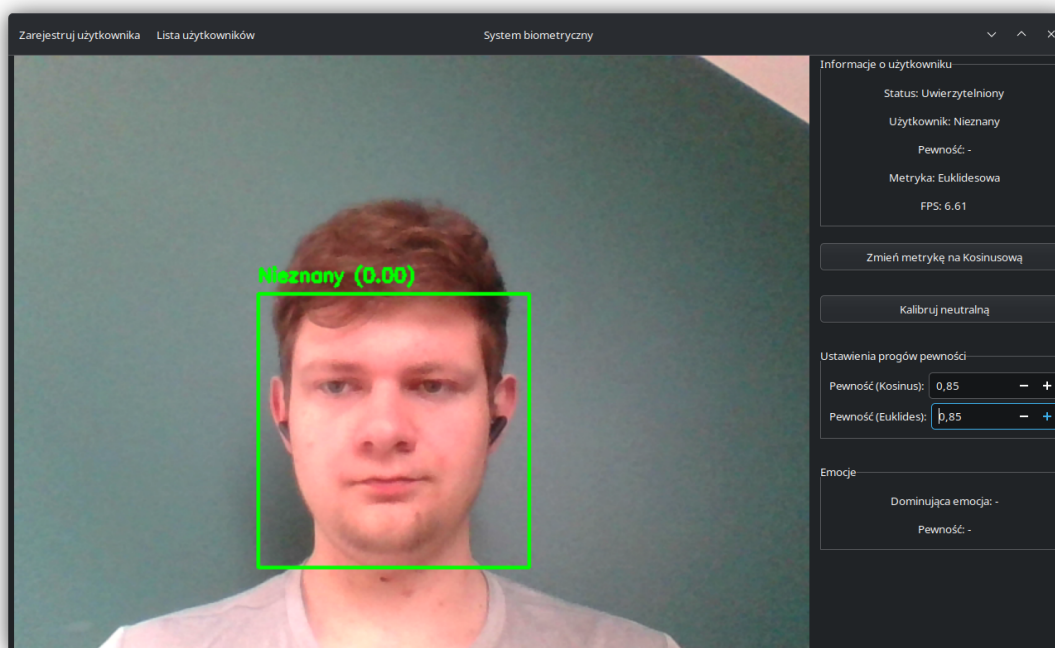
Kluczowym parametrem jest próg tolerancji, który decyduje, czy dwie twarze należą do tej samej osoby. Im niższa wartość odległości, tym twarze są bardziej podobne. Progi te zostały dobrane eksperymentalnie i są zdefiniowane w pliku `face_utils.py`, jak pokazano na listingu 3.1.

Listing 3.1. Domyślne progi tolerancji w pliku `face_utils.py`

```
# Ustawienie progów tolerancji dla różnych metryk
self.tolerances = {
    DistanceMetric.COSINE: 0.15, # Domyślna tolerancja (odpowiada 0.85 pewności)
    DistanceMetric.EUCLIDEAN: 0.35 # Domyślna tolerancja (odpowiada 0.65 pewności)
```

Użytkownik zostaje uznany za dopasowanego, jeśli obliczona odległość jest mniejsza lub równa zdefiniowanemu progowi. Dla większej intuicyjności, w interfejsie graficznym prezentowany jest procentowy wskaźnik **pewności**, obliczany jako $(1 - \text{odległość}) \times 100\%$. Oznacza to, że domyślny próg odległości kosinusowej 0.15 odpowiada wymaganej pewności na poziomie 85%.

Wartość progu jest kluczowa dla bezpieczeństwa – zbyt wysoki próg odległości (niski próg pewności) może prowadzić do fałszywych pozytywów (rozpoznanie niewłaściwej osoby), a zbyt niski próg odległości (wysoki próg pewności) do fałszywych negatywów (nierozpoznanie zarejestrowanego użytkownika). Na rysunku 3.1 widać sytuację, w której twarz została wykryta, ale uzyskana pewność była niższa od wymaganego progu, więc użytkownik nie został zidentyfikowany.



Rys. 3.1. Interfejs w momencie wykrycia twarzy, która nie została dopasowana do żadnego zarejestrowanego użytkownika.

3.2.2. Optymalizacja wydajności poprzez śledzenie obiektów

Pełne rozpoznawanie twarzy w każdej klatce wideo jest operacją kosztowną obliczeniowo. Aby zapewnić płynne działanie aplikacji, zaimplementowano mechanizm optymalizacyjny. Polega on na inteligentnym przełączaniu się między dwoma trybami:

1. **Pełne rozpoznawanie:** Wykonywane co `recognition_interval` klatek (domyślnie 30) lub gdy w scenie pojawi się nowa osoba. Polega na detekcji wszystkich twarzy, ekstrakcji ich cech i porównaniu z bazą danych.
2. **Śledzenie (tracking):** Po pomyślnym rozpoznaniu użytkownika, system inicjuje szybki algorytm śledzący (w projekcie użyto `cv2.TrackerKCF_create()`), który w kolejnych klatkach jedynie aktualizuje pozycję ramki otaczającej twarz. Jest to operacja znacznie mniej wymagająca obliczeniowo.

Logika przełączająca między tymi trybami znajduje się w metodzie `authenticate_user` (listing 3.2) i znacząco redukuje obciążenie procesora.

Listing 3.2. Fragment logiki optymalizacyjnej w metodzie `authenticate_user`

```
def authenticate_user(self, frame: np.ndarray) -> Optional[UserSession]:
    """Uwierzytelnia_użytkownika, używając optymalizacji śledzenia."""
    self._update_fps()
    self.frames_since_last_full_recognition += 1
    face_results = []

    # Najpierw spróbuj zaktualizować istniejące trackery
    if self.active_trackers:
        face_results = self._update_trackers(frame)

    # Jeśli nie ma śledzonych twarzy lub nadszedł czas na pełne skanowanie
    if not self.active_trackers or self.frames_since_last_full_recognition >= self.
        recognition_interval:
        self.frames_since_last_full_recognition = 0
        # Optymalizacja: skalowanie klatki przed detekcją
        small_frame = cv2.resize(frame, (0, 0), fx=0.5, fy=0.5)
        face_results = self._run_full_recognition(frame, small_frame)

    # Dalsza logika pozostaje taka sama
    session = self.authenticate_user_with_results(frame, face_results)
    if session:
        session.face_results = face_results
```

3.3. Autorski algorytm analizy emocji

Moduł `emotion_analyzer.py` stanowi w całości autorskie rozwiązanie problemu analizy emocji. Mimo iż do detekcji siatki punktów na twarzy wykorzystano bibliotekę `MediaPipe`, cała interpretacja tych danych jest zaimplementowana od podstaw.

3.3.1. Kalibracja spersonalizowanego modelu

Kluczową implementacją jest mechanizm kalibracji, który pozwala dostosować system do unikalnej anatomii twarzy każdego użytkownika. System nie opiera się na uniwersalnym modelu emocji, lecz tworzy spersonalizowaną **linię bazową (baseline)**.

Proces kalibracji, uruchamiany przez użytkownika przyciskiem *Calibrate neutral face*, polega na zebraniu określonej liczby próbek (domyślnie 25) jego neutralnego wyrazu twarzy. Dla każdej próbki obliczany jest zestaw cech geometrycznych (opisanych w kolejnej sekcji). Po zebraniu wszystkich próbek system oblicza ich średnią, która staje się punktem odniesienia (linią bazową) dla tego użytkownika. Proces ten jest implementowany w metodzie `start_calibration` w klasie `EmotionAnalyzer`.

Linia bazowa jest następnie zapisywana w sesji użytkownika, dzięki czemu kalibrację wystarczy przeprowadzić tylko raz. Takie podejście sprawia, że system jest znacznie bardziej odporny na indywidualne różnice w budowie twarzy i precyzyjniej wykrywa subtelne zmiany w mimice.

3.3.2. Geometryczna analiza cech

Emocje są rozpoznawane na podstawie zmian w geometrii twarzy. Zaimplementowano funkcję `_calculate_distances`, która oblicza znormalizowane odległości i stosunki między punktami charakterystycznymi, odpowiadające konkretnym ruchom mięśni twarzy, np.:

- **Uśmiech:** Stosunek odległości między kącikami ust a szerokością ust.

- **Zdziwienie:** Stopień otwarcia oczu i ust.
- **Złość:** Odległość między brwiami (zmarszczenie).

3.3.3. Wnioskowanie oparte na regułach i progach

Ostateczna klasyfikacja emocji odbywa się za pomocą autorskiego systemu regułowego. Najpierw obliczane są **odchyłki** bieżących cech geometrycznych od zapisanej linii bazowej. Te odchyłki są normalizowane, aby uniezależnić je od wielkości twarzy w kadrze.

Następnie znormalizowane wartości są mapowane na poszczególne kategorie emocji za pomocą predefiniowanych wag, przechowywanych w słowniku `EMOTION_WEIGHTS` (listing 3.3). Każda cecha (np. 'smile') wpływa na wynik jednej lub więcej emocji (np. 'HAPPY').

Listing 3.3. Fragment wag emocji w pliku `emotion_analyzer.py`

```
# Wagi dla różnych emocji (można dostosować)
self.EMOTION_WEIGHTS = {
    'smile': {'HAPPY': 0.7, 'NEUTRAL': 0.3},
    'surprise': {'SURPRISED': 0.9, 'FEAR': 0.1},
    'brow_furrow': {'ANGRY': 0.6, 'SAD': 0.4},
    'eye_open': {'SURPRISED': 0.7, 'FEAR': 0.3},
    'mouth_open': {'SURPRISED': 0.8, 'HAPPY': 0.2},
    'lip_corner_depression': {'SAD': 0.35, 'ANGRY': 0.2, 'NEUTRAL': 0.45},
    'nose_wrinkle': {'DISGUSTED': 0.9, 'ANGRY': 0.1}
```

Wynikiem jest rozkład prawdopodobieństwa dla każdej z emocji. Emocja z najwyższym wynikiem jest prezentowana w interfejsie użytkownika, co widać na rys. 2.3. Takie podejście, choć prostsze od głębokich sieci neuronowych, jest w pełni interpretowalne i niezwykle wydajne, co jest kluczowe w zastosowaniach czasu rzeczywistego.

3.3.4. Implementacja analizy geometrycznej

Implementacja funkcji `_calculate_distances` (listing 3.4) z modułu `emotion_analyzer.py` oblicza kluczowe cechy geometryczne na podstawie siatki twarzy, co stanowi fundament dla systemu regułowego wnioskującego o emocjach.

Listing 3.4. Implementacja funkcji `_calculate_distances`

```
def _calculate_distances(self, landmarks, frame_shape: Tuple[int, int]) -> Dict[str, float]:
    """Oblicza odległości między punktami charakterystycznymi twarzy."""
    # Pobranie punktów
    p = lambda idx: self._get_landmark_point(landmarks, idx, frame_shape)

    # Obliczenie odległości
    distances = {}

    # Uśmiech - szerokość ust
    distances['smile'] = np.linalg.norm(np.array(p(self.LIPS_RIGHT)) - np.array(p(self.LIPS_LEFT)))

    # Otwarcie ust
    distances['mouth_open'] = np.linalg.norm(np.array(p(self.MOUTH_BOTTOM)) - np.array(p(self.MOUTH_TOP)))
```

```

# Uniesienie brwi
brow_left = p(self.BROW_LEFT)
brow_right = p(self.BROW_RIGHT)
eye_left = p(self.EYE_LEFT)
eye_right = p(self.EYE_RIGHT)

# Średnia odległość między brwiami a oczami
distances['brow_raise'] = ((brow_left[1] - eye_left[1]) + (brow_right[1] -
    eye_right[1])) / 2

# Zmarszczenie brwi
nose_tip = p(self.NOSE_TIP)
distances['brow_furrow'] = (np.linalg.norm(np.array(brow_left) - np.array(
    nose_tip)) +
    np.linalg.norm(np.array(brow_right) - np.array(
        nose_tip))) / 2

# Otwarcie oczu (uproszczone)
distances['eye_open'] = (eye_left[1] - brow_left[1] + eye_right[1] - brow_right
    [1]) / 2

# Opadanie kącików ust
lip_left = p(self.LIPS_LEFT)
lip_right = p(self.LIPS_RIGHT)
distances['lip_corner_depression'] = (lip_left[1] + lip_right[1]) / 2 - (
    brow_left[1] + brow_right[1]) / 2

# Marszczenie nosa (uproszczone)
distances['nose_wrinkle'] = np.linalg.norm(np.array(p(19)) - np.array(nose_tip))

```

3.3.5. Wątek przechwytywania wideo

Klasa `CaptureThread` (listing 3.5) działa w oddzielnym wątku, odpowiadając za pobieranie klatek z kamery i delegowanie zadań rozpoznawania. Takie podejście zapobiega blokowaniu interfejsu graficznego, zapewniając płynność działania aplikacji.

Listing 3.5. Klasa `CaptureThread` z pliku `main.py`

```

# --- Wątek do przechwytywania klatek i rozpoznawania twarzy ---
class CaptureThread(Thread):
    """Oddzielny wątek pobierający klatki z kamery i wykonujący rozpoznawanie.

    Dzięki temu główny wątek GTK pozostaje responsywny."""

    def __init__(self, system: 'BiometricSystem', camera_id: int = 0):
        super().__init__(daemon=True)
        self.system = system
        self.camera_id = camera_id
        self.running = True
        self.frame_lock = Lock()
        self.latest_frame: Optional[np.ndarray] = None
        self.latest_session: Optional[UserSession] = None

    def run(self):
        cap = cv2.VideoCapture(self.camera_id)
        while self.running:
            ret, frame = cap.read()

```

```

    if not ret:
        continue
    # Wykonaj rozpoznawanie w tym wątku
    session = self.system.authenticate_user(frame)
    # Zapisz wyniki w sposób bezpieczny wątkowo
    with self.frame_lock:
        self.latest_frame = frame
        self.latest_session = session
    cap.release()

def stop(self):
    self.running = False

```

3.3.6. Rejestracja nowych użytkowników

Metoda `register_face` (listing 3.6) odpowiada za przetworzenie obrazu, wyekstrahowanie wektora cech twarzy i zapisanie go w bazie danych w celu umożliwienia przyszłego uwierzytelniania.

Listing 3.6. Metoda `register_face` z pliku `face_utils.py`

```

def register_face(self, image: np.ndarray, name: str) -> bool:
    """Rejestruje nową twarz w systemie.

    Args:
        image: Obraz wejściowy w formacie BGR
        name: Identyfikator osoby na zdjęciu

    Returns:
        bool: True jeśli udało się zarejestrować twarz, False w przeciwnym wypadku
    """
    logger.info(f"Rozpoczęcie rejestracji użytkownika: {name}")
    try:
        # Optymalizacja: skalowanie klatki przed detekcją
        small_frame = cv2.resize(image, (0, 0), fx=0.5, fy=0.5)
        rgb_image = cv2.cvtColor(small_frame, cv2.COLOR_BGR2RGB)
        logger.debug("Konwersja obrazu z BGR do RGB i skalowanie zakończone")

        # Wykrywanie położenia twarzy na mniejszym obrazie
        logger.debug("Wykrywanie twarzy...")
        face_locations_small = face_recognition.face_locations(
            rgb_image,
            model='hog',
            number_of_times_to_upsample=self.upsample
        )

        if not face_locations_small:
            logger.warning("Nie wykryto twarzy na zdjęciu")
            return False

        # Przeskaluj lokalizacje z powrotem do oryginalnego rozmiaru
        face_locations = [
            (top*2, right*2, bottom*2, left*2) for (top, right, bottom, left) in
            face_locations_small
        ]

        # Użyjemy pełnowymiarowego obrazu do ekstrakcji cech dla lepszej jakości
        rgb_image_full = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

```

```

logger.debug(f"Znaleziono_{len(face_locations)}_twarzy_na_zdjęciu")

if not face_locations:
    logger.warning("Nie_wykryto_twarzy_na_zdjęciu")
    return False

# Wybieramy największą twarz na zdjęciu
face_locations = sorted(face_locations,
                        key=lambda loc: (loc[2]-loc[0])*(loc[1]-loc[3]),
                        reverse=True)

# Ograniczamy się do jednej twarzy
face_locations = face_locations[:1]

# Pobranie cech twarzy z dodatkowymi parametrami
logger.debug("Ekstrakcja_cech_twarzy...")
face_encodings = face_recognition.face_encodings(
    rgb_image_full,
    known_face_locations=face_locations,
    num_jitters=self.num_jitters,
    model='small' # Użyj szybszego modelu dla wideo
)
logger.debug(f"Wyodrębniono_cechy_dla_{len(face_encodings)}_twarzy")

if not face_encodings:
    logger.warning("Nie_udało_się_wyodrębnić_cech_twarzy")
    return False

# Sprawdzenie czy taka twarz już istnieje
for i, face_encoding in enumerate(face_encodings):
    if self.known_face_encodings: # Tylko jeśli mamy już jakieś twarze w
        bazie
        logger.debug(f"Porównywanie_twarzy_{i+1}_z_bazą_{len(self.
            known_face_encodings)}_znanych_twarzy...")
        face_distances = face_recognition.face_distance(
            self.known_face_encodings,
            face_encoding
        )

        # Znajdź najlepsze dopasowanie
        best_match_index = np.argmin(face_distances)
        best_distance = face_distances[best_match_index]

        logger.debug(f"Najlepsze_dopasowanie:_{self.known_face_names[
            best_match_index]}_"
            f"z_odległością_{best_distance:.4f}_(próg:_{self.
            tolerance})")

        if best_distance <= self.tolerance:
            logger.info(f"Twarz_już_zarejestrowana_jako:_{self.
                known_face_names[best_match_index]}_"
                f"(odległość:_{best_distance:.4f})")
            return False

# Dodanie nowej twarzy
self.known_face_encodings.extend(face_encodings)
self.known_face_names.extend([name] * len(face_encodings))

```

```
logger.info(f"Dodano_nowa_twarz_do_bazy_(łącznie:{len(self.known_face_names
    )}_twarzy)")

# Zapisanie zaktualizowanej listy twarzy
self._save_known_faces()
logger.info(f"Pomyślnie_zarejestrowano_nowa_twarz:{name}")
return True

except Exception as e:
    logger.error(f"Błąd_podczas_rejestracji_twarzy:{e}", exc_info=True)
```


4. Wyniki i analiza

W niniejszym rozdziale zaprezentowano kluczowe wyniki uzyskane podczas testowania zaimplementowanego systemu. Główny nacisk położono na ocenę skuteczności autorskiego systemu zarządzania sesją i danymi biometrycznymi, który stanowił centralny element projektu. Analizie poddano zarówno poprawność działania mechanizmów zapisu i odczytu danych, jak i wydajność wynikającą z zastosowanych rozwiązań.

4.1. Skuteczność autorskiego systemu zarządzania danymi

Kluczowym celem projektu było stworzenie niezależnego mechanizmu do obsługi danych biometrycznych użytkowników. W ramach testów potwierdzono jego pełną funkcjonalność:

- **Trwałość sesji użytkownika:** Zaimplementowana klasa `UserSession` oraz mechanizmy serializacji do formatu JSON działają poprawnie. Po ponownym uruchomieniu aplikacji, system prawidłowo wczytuje zapisane dane, takie jak wektory cech twarzy oraz spersonalizowane linie bazowe dla emocji. Umożliwia to zachowanie ciągłości pracy bez potrzeby ponownej rejestracji czy kalibracji.
- **Personalizacja analizy emocji:** Potwierdzono, że mechanizm kalibracji neutralnego wyrazu twarzy i tworzenia indywidualnej linii bazowej działa zgodnie z założeniami. Umożliwia to systemowi adaptację do unikalnej mimiki użytkownika, co stanowi podstawę do spersonalizowanej i potencjalnie trafniejszej analizy emocji.

Poprawna obsługa danych biometrycznych stanowi fundament dla niezawodności i użyteczności całego systemu.

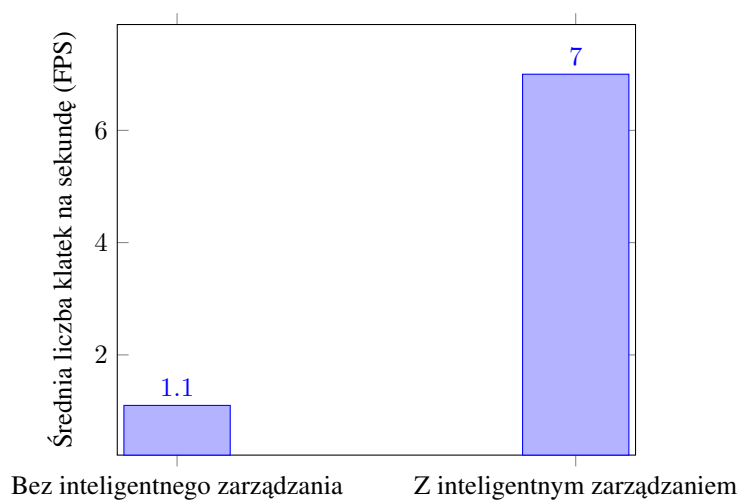
4.2. Analiza wydajności jako efekt inteligentnego zarządzania sesją

Wysoka wydajność aplikacji w czasie rzeczywistym nie była celem samym w sobie, lecz wynikiem zastosowania inteligentnej logiki zarządzania stanem. Zamiast przeprowadzać kosztowne obliczeniowo operacje w każdej klatce, system, opierając się na informacjach z sesji użytkownika, decyduje, kiedy należy wykonać pełne rozpoznawanie, a kiedy wystarczy lżejsze śledzenie.

Aby zobrazować korzyści płynące z tego podejścia, przeprowadzono pomiary średniej liczby klatek na sekundę (FPS) w dwóch scenariuszach:

- **Bez inteligentnego zarządzania:** System w każdej klatce wideo wykonuje pełną detekcję i próbę rozpoznania wszystkich twarzy. W tym trybie uzyskano wydajność na poziomie 0.7-1.5 FPS.
- **Z inteligentnym zarządzaniem:** System wykorzystuje śledzenie obiektów i wykonuje pełne rozpoznawanie tylko w zdefiniowanych interwałach, zarządzanych przez logikę sesji. Ten tryb pozwolił osiągnąć wydajność rzędu 6-8 FPS.

Wyniki pomiarów, uśrednione dla czytelności, zostały przedstawione na wykresie 4.1.



Rys. 4.1. Porównanie średniej wydajności systemu. Wysoka liczba klatek na sekundę jest bezpośrednim wynikiem inteligentnego zarządzania operacjami rozpoznawania w ramach sesji użytkownika.

5. Podsumowanie

Celem niniejszego projektu było zaprojektowanie i zaimplementowanie kompletnego systemu biometrycznego, którego fundamentem jest autorski mechanizm zarządzania danymi i sesją użytkownika. Cel ten został w pełni osiągnięty. Stworzono aplikację, która nie tylko realizuje funkcje rozpoznawania twarzy i analizy emocji, ale przede wszystkim czyni to w oparciu o samodzielnie opracowaną logikę obsługi danych, co stanowiło główne wyzwanie inżynierskie.

Najważniejszym rezultatem pracy jest spójny system, w skład którego wchodzi:

- Moduł do rejestracji i uwierzytelniania użytkowników na podstawie wektorów cech twarzy, przechowyjący dane w lokalnej bazie.
- Autorski system zarządzania sesją, który zapewnia trwałość danych (m.in. zapisanych wektorów cech i skalibrowanych linii bazowych emocji) pomiędzy uruchomieniami aplikacji.
- Mechanizm spersonalizowanej analizy emocji, który dzięki indywidualnej kalibracji dostosowuje się do mimiki konkretnej osoby.

Płynne działanie aplikacji w czasie rzeczywistym, uzyskane dzięki inteligentnemu przełączaniu między trybami rozpoznawania i śledzenia, nie było celem samym w sobie, lecz naturalną konsekwencją przyjętej architektury zorientowanej na zarządzanie stanem sesji. To właśnie logika biznesowa decyduje, jakie operacje są w danym momencie niezbędne, co prowadzi do optymalnego wykorzystania zasobów.

Zaprojektowany system, dzięki swojej modułowej budowie i elastyczności, posiada szeroki potencjał praktycznych zastosowań. Może stanowić fundament dla zaawansowanych systemów kontroli dostępu. Innym obszarem zastosowań są inteligentne interfejsy człowiek-komputer (HCI), które mogłyby dynamicznie adaptować swoje działanie do nastroju użytkownika, np. w systemach e-learningowych lub aplikacjach terapeutycznych.

Praca stanowi również solidną podstawę do dalszego rozwoju. Możliwe kierunki obejmują:

- Rozszerzenie systemu o dodatkowe modalności biometryczne, takie jak rozpoznawanie głosu czy analiza tętnośki, w celu stworzenia wielomodalnego systemu uwierzytelniania.
- Przeprowadzenie formalnych testów użyteczności (UX) oraz skuteczności systemu w warunkach zbliżonych do rzeczywistych, z udziałem zróżnicowanej grupy użytkowników.
- Rozbudowę interfejsu graficznego o zaawansowane wizualizacje danych biometrycznych i wyników analizy emocji w czasie rzeczywistym.

Bibliografia

- [1] Davis E. King. Dlib-ml: A machine learning toolkit. In *Journal of Machine Learning Research*, volume 10, pages 1755–1758, 2009.
- [2] Camillo Lugaresi, Jiuqiang Tang, Hadon Nash, Chris McClanahan, Esha Uboweja, Michael Hays, Fan Zhang, Chuo-Ling Chang, Ming Yong, Juhyun Lee, Wan-Teh Chang, Wei Hua, Manfred Georg, and Matthias Grundmann. MediaPipe: A framework for building perception pipelines. *arXiv preprint arXiv:1906.08172*, 2019.
- [3] OpenCV team. Open Source Computer Vision Library. <https://opencv.org>, 2023.

Spis rysunków

2.1	Schemat blokowy architektury systemu biometrycznego.	8
2.2	Okno dialogowe wyboru kamery.	9
2.3	Główny interfejs aplikacji z zidentyfikowanym użytkownikiem.	10
2.4	Proces rejestracji nowego użytkownika.	10
3.1	Interfejs w momencie wykrycia twarzy, która nie została dopasowana do żadnego zarejestrowanego użytkownika.	12
4.1	Porównanie średniej wydajności systemu. Wysoka liczba klatek na sekundę jest bezpośrednim wynikiem inteligentnego zarządzania operacjami rozpoznawania w ramach sesji użytkownika.	20

Spis listingów

3.1	Domyślne progi tolerancji w pliku <code>face_utils.py</code>	11
3.2	Fragment logiki optymalizacyjnej w metodzie <code>authenticate_user</code>	13
3.3	Fragment wag emocji w pliku <code>emotion_analyzer.py</code>	14
3.4	Implementacja funkcji <code>_calculate_distances</code>	14
3.5	Klasa <code>CaptureThread</code> z pliku <code>main.py</code>	15
3.6	Metoda <code>register_face</code> z pliku <code>face_utils.py</code>	16

Załącznik nr 2 do Zarządzenia nr 228/2021 Rektora Uniwersytetu Rzeszowskiego z dnia 1 grudnia 2021 roku w sprawie ustalenia procedury antyplagiatowej w Uniwersytecie Rzeszowskim

OŚWIADCZENIE STUDENTA O SAMODZIELNOŚCI PRACY

.....Piotr Pająk.....
Imię (imiona) i nazwisko studenta

Wydział Nauk Ścisłych i Technicznych

.....Informatyka.....
Nazwa kierunku

.....131483.....
Numer albumu

1. Oświadczam, że moja praca projektowa pt.: Projekt Zaliczeniowy: System biometryczny oparty na rozpoznawaniu twarzy i analizie emocji
 - 1) została przygotowana przeze mnie samodzielnie*,
 - 2) nie narusza praw autorskich w rozumieniu ustawy z dnia 4 lutego 1994 roku o prawie autorskim i prawach pokrewnych (t.j. Dz.U. z 2021 r., poz. 1062) oraz dóbr osobistych chronionych prawem cywilnym,
 - 3) nie zawiera danych i informacji, które uzyskałem/am w sposób niedozwolony,
 - 4) nie była podstawą otrzymania oceny z innego przedmiotu na uczelni wyższej ani mnie, ani innej osobie.
2. Jednocześnie wyrażam zgodę/nie wyrażam zgody** na udostępnienie mojej pracy projektowej do celów naukowo-badawczych z poszanowaniem przepisów ustawy o prawie autorskim i prawach pokrewnych.

(miejscowość, data)

(czytelny podpis studenta)

* Uwzględniając merytoryczny wkład prowadzącego przedmiot

** – niepotrzebne skreślić

Załącznik nr 2 do Zarządzenia nr 228/2021 Rektora Uniwersytetu Rzeszowskiego z dnia 1 grudnia 2021 roku w sprawie ustalenia procedury antyplagiatowej w Uniwersytecie Rzeszowskim

OŚWIADCZENIE STUDENTA O SAMODZIELNOŚCI PRACY

.....Mateusz Czapor.....

Imię (imiona) i nazwisko studenta

Wydział Nauk Ścisłych i Technicznych

.....Informatyka.....

Nazwa kierunku

.....131416.....

Numer albumu

1. Oświadczam, że moja praca projektowa pt.: Projekt Zaliczeniowy: System biometryczny oparty na rozpoznawaniu twarzy i analizie emocji
 - 1) została przygotowana przeze mnie samodzielnie*,
 - 2) nie narusza praw autorskich w rozumieniu ustawy z dnia 4 lutego 1994 roku o prawie autorskim i prawach pokrewnych (t.j. Dz.U. z 2021 r., poz. 1062) oraz dóbr osobistych chronionych prawem cywilnym,
 - 3) nie zawiera danych i informacji, które uzyskałem/am w sposób niedozwolony,
 - 4) nie była podstawą otrzymania oceny z innego przedmiotu na uczelni wyższej ani mnie, ani innej osobie.
2. Jednocześnie wyrażam zgodę/nie wyrażam zgody** na udostępnienie mojej pracy projektowej do celów naukowo-badawczych z poszanowaniem przepisów ustawy o prawie autorskim i prawach pokrewnych.

(miejscowość, data)

(czytelny podpis studenta)

* Uwzględniając merytoryczny wkład prowadzącego przedmiot

** – niepotrzebne skreślić